Code Jam to I/O for Women 2019 - Code Jam to I/O for Women

Analysis: Grid Escape

Test set 1

In the first test set, there can be at most 8 rooms in the grid. Each room can have its working exit in any one of the four directions, so we can use brute force to check all 4^8 possible ways to choose the 8 working exits, and see how many players escape from each possible layout. Either we will find a solution, or we will show that the case is `IMPOSSIBLE`.

Test set 2

In the second test set, we can no longer get away with an exhaustive approach! However, since we have some freedom in designing the grid, it's worth looking for a constructive solution.

If **R** = **C** = 1, then **K** = 0 is impossible, and any solution works for **K** = 1. Otherwise, imagine drawing a "snaking" path through all of the rooms of the grid, choosing our working doors to allow us to follow this path. We start in the northwesternmost cell of the grid, and move eastward through rooms until we reach the northeasternmost cell. (If **C** = 1, we will already be there and so we will not need to move.) Then we make one move south, and if we have not exited the grid, we move westward through rooms until we reach a cell on the western border. Then we make one more move south (if possible), and then move eastward, and so on, until one of our moves to the south exits the grid.

(By the way, a more obscure term for this type of winding path is *boustrophedon*, which derives from a way to direct oxen when plowing a field. Algorithms have been useful for a very long time!)

Notice that all of the players can escape along our path, no matter where they start. Now consider choosing some room along the path and "reversing" its working door — that is, choosing the door that opens onto the previous room on the path. Then every player who starts in that room or a room earlier than it along the path cannot escape — they will be trapped in an infinite loop! Every other player will still be able to escape just as before.

Since we can choose any room along the path, we can allow any desired number of players to escape... well, *almost* any number. If we try to use the above strategy to allow all but one of the players to escape, it breaks down because the very first room (where that player is) has no "previous room on the path".

In fact, we can never allow all but one of the players to escape. Consider the room with the one player who we want to prevent from escaping. One of their doors has to work, and it cannot open onto the outside world, so it must open onto another room. But that means that our player will share the fate of the player in that other room — either they will both be able to escape, or neither will.

So, if **K** = **R** × **C** - 1, the case is `IMPOSSIBLE`, and for any other case, the above construction gets us our answer. If **K** = **R** × **C**, we do not modify any of the pathâ€™s doors; otherwise, to allow exactly **K** of the players to escape, we "reverse" the door in the (**K**+1)-th-from-last room of the path.

Many other approaches are possible. For example, we can design around the target number of players who cannot escape. As long as that number is at least two, we can create a "nucleus" of the trap in the upper left corner of the grid: either an `E` room to the west of a `W` room, or an `S`

room to the north of an `N` room, depending on our given dimensions. Then we can turn cells sharing rows with the nucleus into `W`s, cells sharing columns with the nucleus into `N`s, and other cells (scanning top-to-bottom, then left-to-right) into `W`s until our set of trapped rooms is large enough. The remaining rooms can safely be filled with `S`. Alternatively, we can use flood fill to extend the sink.

Code Jam to I/O for Women 2019 - Code Jam to I/O for Women

Analysis: Parcel Posts

Test set 1

Test set 1 has a small upper bound of only 10 for **K**, which leaves at most 9 places where a post can be placed. That means we can just try every one of the $2^9 = 512$ combinations. For each one, check whether all of the produced parcels are valid, and if they all are, update a result if the number of posts in this option is larger than it. To check whether a parcel is valid, notice the condition is equivalent to checking if the list of elevations is not sorted non-decreasingly nor non-increasingly. We can do that by either sorting it in both ways and compare whether either is equal to the original, or check there exist two consecutive marks with one strictly lower than the other, and also two consecutive marks with one stricly higher than the other.

Test set 2

We can improve upon the exponential time required for the solution in the previous paragraph with the following insight: if a parcel between marks i and j is valid, so is any extension of it to a parcel between marks i-d and j+e for non-negative d and e. So, let i be the westernmost mark such that the parcel between marks 0 and i is valid and the parcel between i and **K** is valid. If there is not such an i, then the result is just 0 (no posts can be added). We can then prove that there is a way to place a maximum number of posts that places a post at mark i and not at any mark west of i. If that's true, a greedy algorithm follows: find such i, place a post there, and recursively try to place more posts within the [i, **K**] parcel. This algorithm takes $O(\mathbf{K}^2)$ time if implemented as described, which is fast enough to solve this test set, but a linear time alternative is presented in the next paragraph.

To solve the problem in linear time, we scan the list of elevations from west to east, recording when we see consecutive marks in strictly increasing and strictly decreasing order. As soon as we have seen both, we found a potential i (we don't check if [i, **K**] is valid just yet, to save time). We add 1 to the result, reset whether we have seen markings in each order to false, and resume scanning (this is equivalent to the recursive step in the previous presentation). After the scan is complete, it's possible that the eastmost parcel is not valid (since we didn't check it), but we can check it by inspecting the value of the same two boolean variables showing whether we saw markings in each order since our last reset. If the westmost parcel is not valid, the last place we found for a post was not valid, so we subtract 1 from our running result. Notice that each time we find a place where we can place a new post, it means that all previously placed posts were valid. This takes a single linear pass and a constant number of updates and checks of 3 variables at each step (two boolean variables about having seen markings in each order, and the result), so the algorithm runs in linear time.

To prove the main proposition, notice first that the parcel between marks 0 and j for $j < i$ is never valid by definition of i, so any valid placement of posts does not place a post at mark $j < i$. For the second part, since just i is a valid placement, there exists a non-empty valid placement. Let $L = i_1, i_2, ..., i_m$ be the list of mark positions where posts should be added in a valid placement of a maximum number m of posts, in increasing order. Then, $i \leq i_1$ because of the first part, so we can define $L' = i, i_2, ..., i_m$. Now we can show L' is also such a list, and one that has i as its westernmost post. L' has m posts by definition. The validity of L' follows because the parcel between marks 0 and i is valid by definition of i, and the parcel between i and i_2 is an extension of the one between i_1 and i_2 (because $i \leq i_1$ from the previous part). All other parcels in L' are also parcels in L, and are therefore also valid, so L' is valid.

Analysis: Sheepwalking

Herding strategy

What is the most efficient way to herd Bleatrix from cell (**X**, **Y**) back to the "origin" cell (0, 0)? We will simplify our discussion by assuming that **X** and **Y** are both nonnegative; because Bleatrix's two-dimensional field is symmetric across both axes, our solutions can safely work with the absolute values of **X** and **Y**.

If Bleatrix is ever at a cell (x, y) such that $x \neq 0$, and $y \neq 0$ — that is, she is not along one of the two "axes" of cells — then two of her possible moves will take her toward the origin cell (reducing either her horizontal or vertical distance from it), and the other two will take her away from the origin. In this case, we should use the two sheepdogs to block the two "away" moves. Any move away from the goal in some direction would need to be reversed later on, costing us two moves. Moreover, we do not want to spend both sheepdogs to block opposite directions of movement (e.g., placing them to Bleatrix's left and right to force her to move either up or down), because then she would make a purely random horizontal or vertical walk, with no tendency toward the goal.

The only remaining case — apart from being at the origin and thus being done — is that Bleatrix is at a cell of the form (x, 0) or (0, y). That is, she is along one of the two main "axes" of unit cells; without loss of generality, we will suppose she is at a cell of the form (0, y).

We know from the argument above that we do not want to use both sheepdogs to force her to randomly walk along the y-axis, so we should use one sheepdog to block her movement along the y-axis away from the origin. But should we deploy the other sheepdog to block one of her moves perpendicular to the axis? If we do not, she will move toward the goal with probability ⅓ and perpendicular to it with probability ⅔; if we do, she will move toward or perpendicular to the goal with equal probability. Since moving toward the goal is better (again, any lateral move needs to be undone eventually), we should use the additional sheepdog. For convenience, we will always deploy it in a way that keeps both of her coordinates nonnegative — that is, below her if she is along the x-axis, and to the left of her if she is on the y-axis.

A more rigorous proof of the optimality of our strategy follows at the end of this analysis.

Simulation can only get us so far!

Now that we have a strategy, a natural approach is to simulate it and take the average of, say, a million runs. Test set 1 only includes nine distinct cases; the others are symmetric, differing only in signs and/or in which values are X and Y. How hard can it be to get those nine answers?

As it turns out: pretty hard! The problem is that the length of our random walk has very high variance, so it is hard to get a confident estimate of the true expected length. A straightforward simulation solution will either take too long or not meet the strict tolerance requirements of the problem. However, we can run simulations offline and inspect the results, and we might notice that the answers are always close to a value of the form A / 9, where A is some integer. This pattern will not hold up for X and Y values outside of the [-3, 3] range, but it can serve us well here — an offline simulation can get us close enough to the true answers that we can confidently guess the value of A for each case, and then submit a solution that packages up those answers.

It is also possible to solve Test set 1 by hand, using the same method that also leads to a solution for Test set 2...

Expected herding time

Notice that the problem is "memoryless"; if Bleatrix moves to cell (x, y), the expected number of additional moves to reach the origin from there is the same as if she had begun at (x, y). So, to calculate the expected number of moves needed from a given starting point, we can use a series of *recurrences*. Let T(x, y) be the expected number of moves when starting from cell (x, y); trivially, T(0, 0) = 0. Let us consider our cases from above:

- Some cell (x, y) not on an axis: Bleatrix will move either left or down with equal probability; either way, this uses one move. So we have T(x, y) = ½ T(x-1, y) + ½ T(x, y-1) + 1.
- Some cell (0, y) on the y-axis: Bleatrix will move either down or right with equal probability; either way, this uses one move. So we have T(0, y) = ½ T(0, y-1) + ½ T(1, y) + 1. (The expression for a cell on the x-axis is similar.)

We can simplify the second recurrence by replacing T(1, y) with its representation from the first recurrence: ½ T(0, y) + ½ T(1, y-1) + 1. Then, after some algebra, the second recurrence simplifies to T(0, y) = ⅔ T(0, y-1) + ⅓ T(1, y-1) + 2. Now we are expressing T(0, y) only in terms of recurrences for smaller values of y. So, all told, for any starting cell, we have T(x, y)=

- 0 if x = y = 0.
- ⅔ T(0, y-1) + ⅓ T(1, y-1) + 2, if x = 0 but y ≠ 0.
- ⅔ T(x-1, 0) + ⅓ T(x-1, 1) + 2, if y = 0 but x ≠ 0.
- ½ T(x-1, y) + ½ T(x, y-1) + 1, if x ≠ 0 and y ≠ 0.

At this point, we can use recursion plus memoization — or some other form of dynamic programming — to quickly compute any answer we want. We will never need to evaluate T() for any particular cell more than once, so we have tamed the infiniteness and randomness inherent in the problem.

If we use, e.g., C++ `double`s, might our solution fail due to accumulated floating-point errors? This phenomenon is often worth worrying about, but in this problem, it is not even close to posing a threat. The total number of calculations needed is not very large — only a few per recurrence. Even if our solution for e.g. the case `500 500` involves calculating the answer for every possible distinct test case along the way, we will have at most a few million chances to introduce a tiny amount of error. A standard double precision floating point number has 15 decimal digits of precision, but we only need the first 6 to be correct, so even if our errors pathologically did not cancel each other out at all, we would be fine.

A (perhaps) surprising property of the solution

Check out the answers for the cases `100 0`, `100 1`, ..., all the way up to `100 30`. They are all (to nine decimal places) the same: 201.500000000. The answer for `100 100` is 212.727275769. Why are these values so close? That is, if we compare starting 200 moves away from the goal at (100, 100) versus starting only 100 moves away from the goal at (100, 0), why does the latter not help us very much?

Observe that, in our strategy, once Bleatrix has made enough left moves to reach the y-axis, her horizontal moves will alternate between right and left; once she has made enough down moves to reach the x-axis, her vertical moves will alternate between up and down. But regardless of where we are in the strategy, she has an equal probability of moving horizontally or vertically. Given that Bleatrix starts at (**X**, **Y**), we can be sure that she will reach the goal only after she has made at least **X** horizontal moves *and* at least **Y** vertical moves.

How many moves will it take until the first time at which both of those conditions are satisfied? Without loss of generality, let us assume that **X** ≥ **Y**. If **X** is much larger than **Y**, then by the time we get our **X**-th horizontal move, we will almost certainly have gotten **Y** vertical moves, so we can ignore **Y**. Then the expected number of moves needed to get **X** horizontal moves is 2**X**, since the probability of a horizontal move is ½. However, if **X** is not much larger than **Y**, or equal to **Y**, we cannot ignore **Y**, and it may take more than 2**X** moves in expectation for *both* conditions to be satisfied.

The above explains why the (100, 100) answer is larger than the (100, 0) answer. It also explains why it is not *too* much larger, since a large discrepancy would imply that it is very likely to get 100 moves in one direction *long* before getting 100 moves in the other direction.

However, why is the (100, 0) answer 201.5 rather than 2**X** = 200? Consider the first time at which we have at least **X** horizontal moves and at least **Y** vertical moves; call the number of moves M. We have just hit one of the axes for the first time, and we will be on the other axis (and thus done) if M matches the parity of **X** + **Y**, or one cell away otherwise. We know that T(0, 1) = T(1, 0) = 3; for large **X** + **Y**, we would expect parity effects to even out, so this final herding process should add ½(3) = 1.5 more moves to the expected total.

Also note that even though our chosen precision level obscures it, the (100, 0) answer is in fact slightly smaller than the (100, 1) answer, and so on.

Appendix: optimality proof

T(0, 0) = 0, and for any other (x_0, y_0), $T(x_0, y_0)$ is the minimum of 1/k × the sum of the T values of any k neighbors of (x_0, y_0), where k ≥ 2. Observe that k can always be 2 — it is never suboptimal to place 2 dogs adjacent to the current cell, since they block the neighbors with the larger values of T, and decrease our average T over all remaining neighbors. Therefore, to compute $T(x_0, y_0)$, we will pick any smallest two among $T(x_0 - 1, y_0)$, $T(x_0 + 1, y_0)$, $T(x_0, y_0 - 1)$, and $T(x_0, y_0 + 1)$, and take their average.

Lemma 1: $T(x_0 - 1, y_0) \le T(x_0 + 1, y_0)$ for all $x_0 > 0$, $y_0 \ge 0$.

Lemma 2: $T(x_0 - 1, y_0) \le T(x_0, y_0 + 1)$ for all $x_0 > 0$, $y_0 \ge 0$.

By Lemmas 1 and 2, $T(x_0 - 1, y_0)$ (and, by symmetry, $T(x_0, y_0 - 1)$) are better than the other two neighbors, so we place the sheepdogs to block the other two neighbors.

Proof of Lemma 1: Let *S* be an optimal strategy starting from $(x_0 + 1, y_0)$. From $(x_0 - 1, y_0)$, we could use a strategy *S*' that, as long as we don't get to the column of cells $x = x_0$, always places the dogs in a way that "mirrors" their placements in *S* with respect to the column $x = x_0$. Call this stage 1. When we first touch that column, we switch to using strategy *S*; call this stage 2.

During stage 1, if strategy *S* is in cell $(x_0 + a, b)$ after k moves, then *S*' is in cell $(x_0 - a, b)$. During stage 2, after k moves, they are always in the same cell. Since the path from $(x_0 + 1, y_0)$ to (0, 0) must cross column $x = x_0$, using this strategy means that for any random choices, either our modified strategy reaches (0, 0) before crossing column $x = x_0$, and therefore before strategy *S*... or the two strategies reach (0, 0) during stage 2, that is, both at the same time.

Sketch of proof of Lemma 2: This is similar to our proof of Lemma 1, but we mirror across the diagonal $y = x_0 + y_0 - x$. From $(x_0, y_0 + 1)$, the path must cross that diagonal, so we can define *S*' with similar stages, and a similar argument demonstrates the inequality.

Code Jam to I/O for Women 2019 - Code Jam to I/O for Women

Analysis: War of the Words

It might seem impossible to promise that we will achieve any particular score in a game in which we do not know which options are better than others, and we don't even know when we are scoring points! But we have our ways...

Test set 1

At first, we might be tempted to send a totally random word every turn. Unfortunately, that's not enough to pass the first test set! When we pick a random word W, the robot will respond with a word R chosen uniformly at random from all words ranked higher than W. Since our W will have a 50th percentile rank on average, the robot's R will have a 75th percentile rank on average, so we might expect to score around 25 points in each case (less if we tie). But we have to get at least 25 points in each of 50 test cases. Good luck winning 50 coin flips in a row!

One approach against an unknown strong opponent, familiar to practitioners of aikido, is to use the opponent's own strength against them:

- Play some fixed but arbitrary word W — say, `HIGCJ`. The robot responds with some word R_1.
- Play W again. The robot responds with some word R_2.
- Now "borrow" and play R_1; the robot responds with some word R_3.
- Play W again. The robot responds with some word R_4.
- Now "borrow" and play R_3; the robot responds with some word R_5...

Suppose that our initial choice of W ranks somewhere around the middle of the pack. We certainly lose by playing W against R_1, since we know that R_1 beats W. But then when we borrow R_1 to play against R_2, we know that both R_1 and R_2 were chosen uniformly and independently at random from among words that beat W, so we should a slightly less than 50% chance of winning (because R_1 and R_2 might be the same, and we do not score points for ties). Then we lose again by playing against R_3, but when we borrow R_3 to play against R_4, we are playing a word that beat a word that beat W against a word that beat W, so we should have around a 75% chance of winning. As this strategy unfolds, we should lose every other round, but win the remaining rounds with probability 50%, 75%, 87.5%, and so on. This seems like it should easily be enough to get us to 25 points.

However, there is a flaw in this argument: as our "borrowed" words get stronger and stronger, at some point, one of them may be the highest-ranking word, in which case the next "borrowed" robot word will be the lowest-ranking word. Then our "borrowed" words, which are supposed to get us our wins, are actually weak and will cause us to lose those rounds. But now we start to win on the rounds on which we play W, since the robot is playing weakish words in response to our weak "borrowed" words, and W probably beats them. So we still get our wins, but now from the W rounds, until the cycle passes by W again. Notice that this implies that the rank of our initial choice of W was not important.

It turns out that this strategy scores close to 50 points on average, but with enough variance that it will not pass test set 2. (We do not need to worry about it failing test set 1, though. In general, running simulations can help us get estimates of the variance of random solutions like this.)

Test set 2

The most notable characteristic of the game is the fact that the otherwise lowest-ranking word L beats the highest-ranking word H. Can we exploit this irregularity? Suppose that we had a way of knowing the identities of L and H. Then we could score a point on *every* turn by repeating the following cycle:

- We play H, forcing the robot to play L in response.
- We play some arbitrary word that is not H or L, beating L and earning a point. The robot plays some other word W in response.
- We beat W with H, forcing the robot to play L in response, and so on. (If W just happened to be H, which is unlikely, then we play L in response instead, and the robot plays another word W' in response, and so on.)

So, if we can confidently find L and H in fewer than 50 turns, we can certainly pass.

One strategy is as follows. Start with an arbitrary word, and then spend a number of turns copying the robot's most recent response. Since the robot always has to play something higher-ranking in response, and our play will always be of the same rank as the robot's last word, this algorithm will gradually visit higher-ranked words. Eventually, there must be an exchange in which we play H and the robot is forced to play L. Then we climb the ranks again from there. So, this strategy will repeatedly "cycle" us through the rankings. Although we will not necessarily visit the same particular subset of ranks each time, each cycle will contain H followed by L.

How many of our turns (not counting the robot's turns) will it take us, on average, to reach the H to L transition? It will take 0 turns if we are lucky enough to start there, 1 turn if we start 1 rank away, an expected 1 + ((0 + 1) / 2) = 1.5 turns if we start 2 ranks away, an expected 1 + ((0 + 1 + 1.5) / 3) = 1.8333... turns if we start 3 ranks away, and so on. These are the harmonic numbers. If we start with a random one of our 10^5 words, the mean number of turns to reach the highest-ranking word is about 11.09. So if we spend 50 turns, we can go through at least three cycles, with high probability.

Then, our program can look through the results to identify these H to L transitions; we should see multiple instances of those two words back to back. There may be a few other instances of consecutive ranks that appear in multiple cycles — for example, maybe the robot always happens to choose the second-highest-ranking word on its way to the highest-ranking word. But the H to L must be present in every cycle, and even if we are unlucky and other consecutive patterns also appear in all of our cycles, we can still reason that the one that occurs closely after all the others is the real transition, since other consecutive patterns are only likely to occur closely before the transition.

Another approach is based on the fact that there are only two words that leave the robot with a single choice: if we play the second-highest-ranking word, the robot must play the highest-ranking word, and if we play the highest-ranking word, the robot must play the lowest-ranking word. So, when the robot sends us a word W, we can send two instances of W and note the robot's two responses. If they are different, we pick the first one and send two instances of that, and so on. If they are the same, we can send W a few more times to see whether we always get the same response R. Then we can send R a few times to see whether we always get the same response S (in which case R is the second-highest-ranking word and S is the highest-ranking word) or varying responses (in which case R is the highest-ranking word and S is the lowest-ranking word).

You can experiment locally and run your own simulations until you are confident that you have a solution that will pass the test sets with high probability. Since both test sets are Visible, and the judge is deterministic, it is possible to tweak your solution and try again if you get unlucky with a reasonable approach. Any overall success probability over, say, 50% should suffice, but the approaches described above should do much better than that.